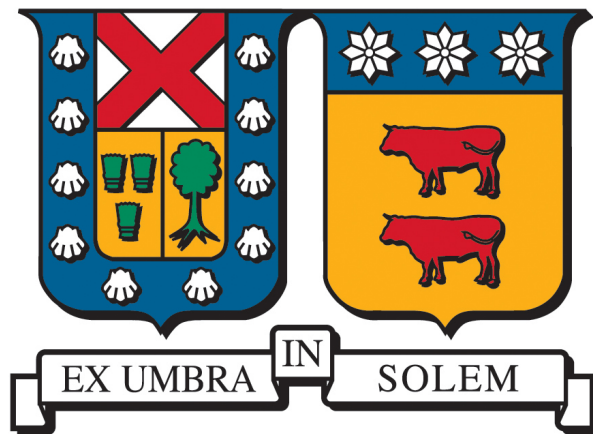




Entrega 2 - Directorio Médico

Debora Alayo Azabache
Ernesto Barría Andrade
Joaquín Gatica Hernández
Sebastián Gutiérrez Milla

Repositorio: <https://github.com/Equipo2PDW/ProyectoDirectorioMedico>

Video: <https://www.youtube.com/watch?v=grC-bNLI4As>



Índice

1. Metodología de trabajo	2
2. Alcance de la herramienta	4
3. Descripción del trabajo realizado	4
3.1. Montar el sistema CI/CD	4
3.2. Actualización del proyecto	5
3.3. Especificar dependencias entre la herramienta y la aplicación	5
4. Uso de Github Actions	5
5. Integraciones	7
6. Problemas encontrados y soluciones	8

1. Metodología de trabajo

En este trabajo utilizamos la metodología Kanban mediante la aplicación Jira, la cual tiene integraciones con GitHub.

Para manejar el repositorio utilizamos Gitflow. Dejamos protegida la branch main para que no fuera posible hacerle push sin una Pull Request primero. Además, los avances grandes los realizábamos en una branch feature para luego hacerle merge a la branch develop.

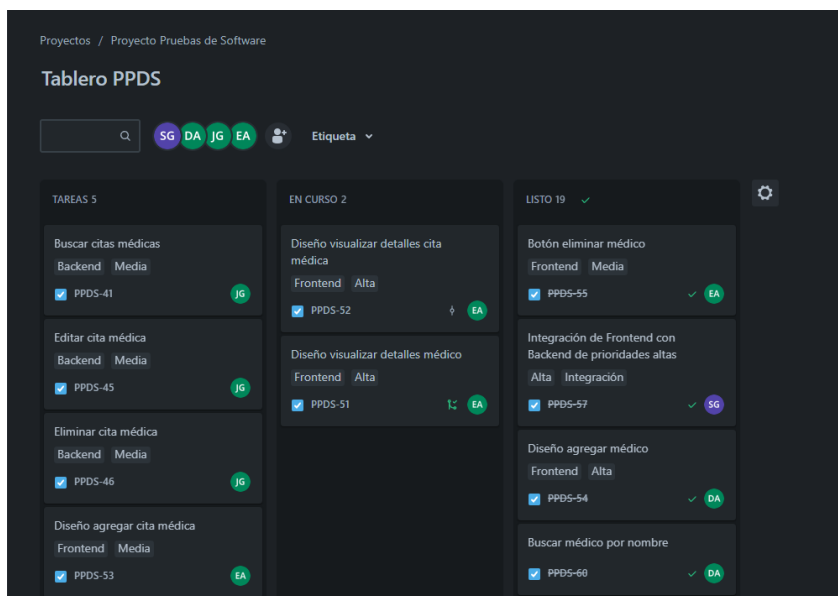


Gráfico 1: Tablero de Jira con nuestros requisitos.

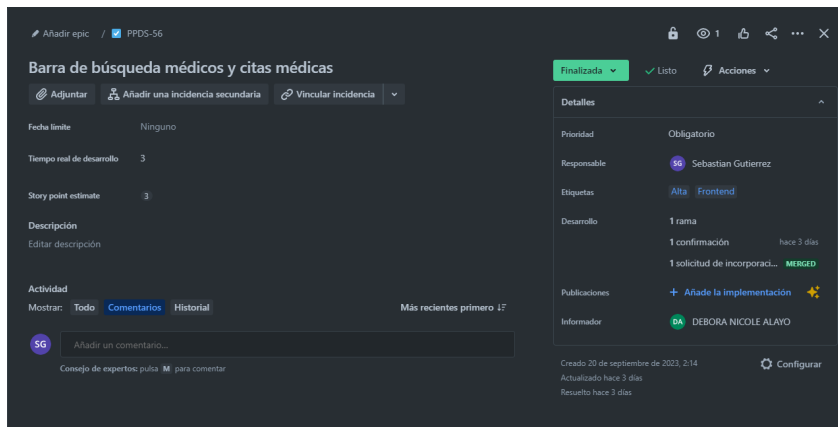


Gráfico 2: Incidencia de Jira con story points e integración de GitHub.

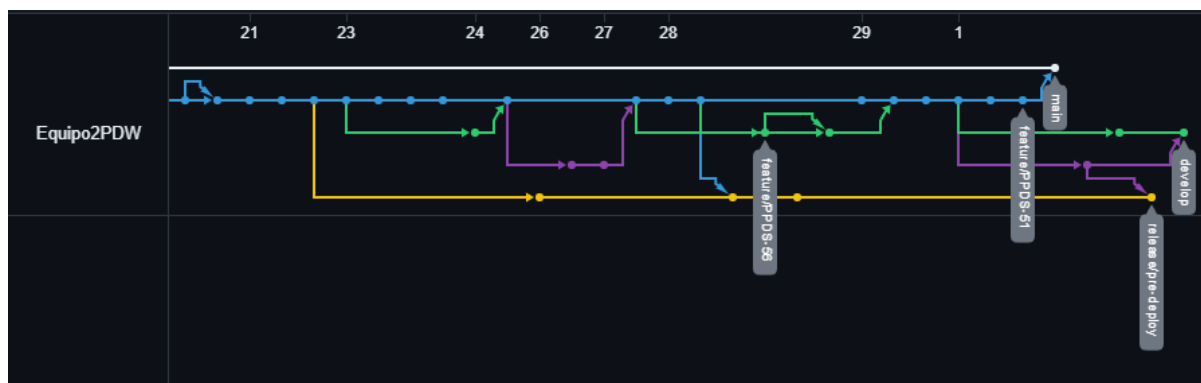


Gráfico 3: Parte del network graph del repositorio con las branch definidas para usar Gitflow.

2. Alcance de la herramienta

Sin duda, la elección de GitHub Actions como nuestra herramienta principal de integración continua y entrega continua (CI/CD) ha resultado clave en la optimización de nuestro flujo de trabajo. Esta solución, que proviene directamente de GitHub, nos permite implementar CI/CD de manera eficiente y sin depender de programas auxiliares adicionales.

En cuanto a su alcance, GitHub Actions sobresale por su capacidad para ejecutar tareas de CI/CD sin complicaciones, ofreciendo una integración perfecta con los flujos de trabajo existentes. Su actions marketplace es otro punto a destacar, ya que proporciona una amplia gama de snippets y workflows predefinidos específicamente diseñados para las tecnologías más populares. Esto no solo agiliza el proceso de desarrollo, sino que también simplifica la creación de CI/CD, permitiéndonos alcanzar nuestras metas de forma más rápida y eficaz.

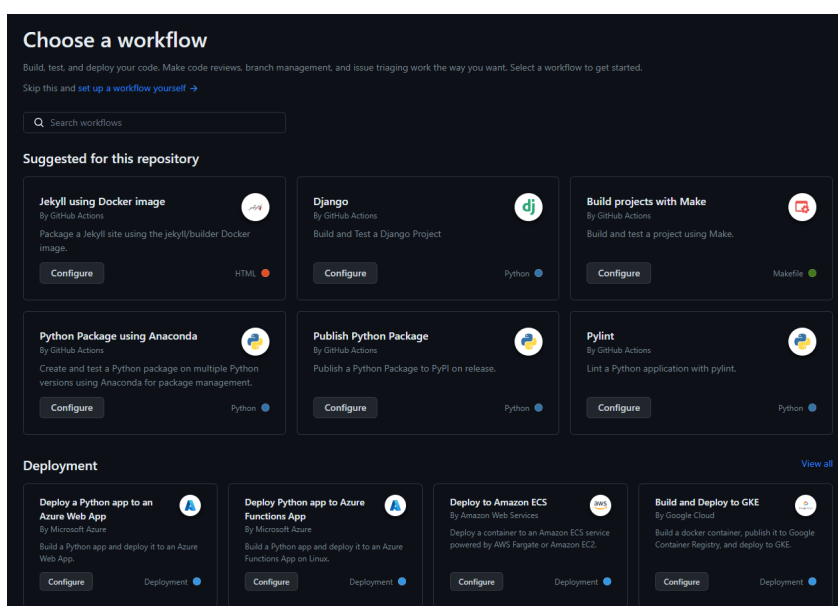


Gráfico 4: Github Actions Marketplace.

3. Descripción del trabajo realizado

3.1. Montar el sistema CI/CD

Este paso es muy fácil, ya que al tener nuestro repositorio en Github, solo tuvimos que ir al apartado de `.Actions` crear los workflows que necesitáramos.

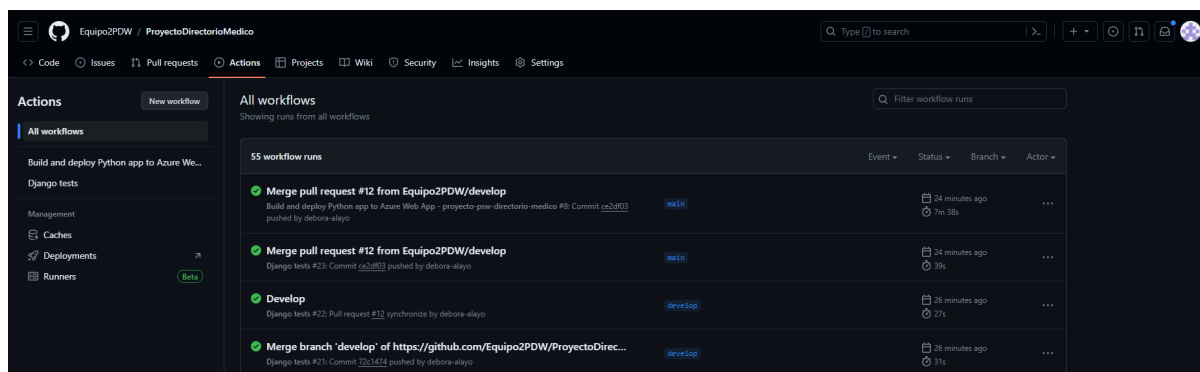


Gráfico 5: Github Actions dentro de Github.

3.2. Actualización del proyecto

Para esta entrega se implementaron 2 funciones nuevas, una para poder agregar citas médicas y otra para poder eliminarlas. Cabe destacar que, en esta ocasión, nos enfocamos más en el backend y funcionalidad que en el css y el frontend.

3.3. Especificar dependencias entre la herramienta y la aplicación

Github actions toma el repositorio donde tenemos nuestra aplicación y crea una máquina virtual en la cual podemos ir indicando en un archivo YAML, todos los snippets y comandos de consolas necesarios para ejecutar nuestros test o lo que se necesita para hacer el deployment de la aplicación en Azure.

4. Uso de Github Actions

Para usar Github Actions, lo que debemos hacer es definir, crear un workflow desde la sección ".Actions" del repositorio o en la carpeta ".github/workflows". agregar los YAML que queramos.

Dentro de este archivo YAML, tenemos la oportunidad de establecer y ajustar diversas propiedades de la máquina virtual que se va a utilizar. Esto incluye detalles esenciales como el sistema operativo requerido, la versión específica del sistema, y la disponibilidad de aplicaciones y entornos como Python, Chrome, entre otros. La capacidad de definir estas propiedades nos permite asegurarnos de que nuestras acciones se ejecuten en un entorno controlado y óptimo para nuestros fines.

Además, este archivo YAML nos permite especificar detalladamente qué acciones deben realizarse y cómo llevarlas a cabo, ya sea a través de la inclusión de snippets predefinidos o la ejecución de comandos de consola específicos.



```
1  name: Django tests
2
3  on:
4    push:
5      branches: [ "main", "develop" ]
6    pull_request:
7      branches: [ "main", "develop" ]
8    workflow_dispatch:
9
10 jobs:
11   build:
12
13     runs-on: ubuntu-22.04
14
15     steps:
16     - uses: actions/checkout@v3
17
18     - name: Set up Python 3.12
19       uses: actions/setup-python@v3
20       with:
21         python-version: 3.12
22
23     - name: Install Dependencies
24       run: |
25         python -m pip install --upgrade pip
26         pip install -r requirements.txt
27
28     - name: Run Tests
29       run: |
30         python manage.py test
31
```

Gráfico 6: YAML del workflow para hacer CI.



```
5 name: Build and deploy Python app to Azure Web App - proyecto-psw-directorio-medico
6
7 on:
8   push:
9     branches:
10       - main
11   workflow_dispatch:
12
13 jobs:
14   build:
15     runs-on: ubuntu-latest
16
17     steps:
18       - uses: actions/checkout@v4
19
20       - name: Set up Python version
21         uses: actions/setup-python@v1
22         with:
23           python-version: '3.9'
24
25       - name: Create and start virtual environment
26         run: |
27           python -m venv venv
28           source venv/bin/activate
29
30       - name: Install dependencies
31         run: pip install -r requirements.txt
32
33       # Optional: Add step to run tests here (Pytest, Django test suites, etc.)
34
35       - name: Zip artifact for deployment
36         run: zip release.zip ./" -r
37
38       - name: Upload artifact for deployment jobs
39         uses: actions/upload-artifact@v3
40         with:
41           name: python-app
42           path: |
43             release.zip
44             !venv/
45
46   deploy:
47     runs-on: ubuntu-latest
48     needs: build
49     environment:
50       name: 'Production'
51       url: ${ steps.deploy-to-webapp.outputs.webapp-url }
52
53     steps:
54       - name: Download artifact from build job
55         uses: actions/download-artifact@v3
56         with:
57           name: python-app
58
59       - name: Unzip artifact for deployment
60         run: unzip release.zip
61
62       - name: 'Deploy to Azure Web App'
63         uses: azure/webapps-deploy@v2
64         id: deploy-to-webapp
65         with:
66           app-name: 'proyecto-psw-directorio-medico'
67           slot-name: 'Production'
68           publish-profile: ${{ secrets.AZUREAPPSERVICE_PUBLISHPROFILE_31A18DA0938342F78589EBE8363C9087 }}
```

Gráfico 7: YAML del workflow para hacer CD.

5. Integraciones

Como se mencionó anteriormente, al ser Github actions parte de Github, la integración de estos 2 fue automática.

Para Slack, al tener todos los avisos posibles de Github en un solo canal, también se incluyeron los Github Actions ahí.

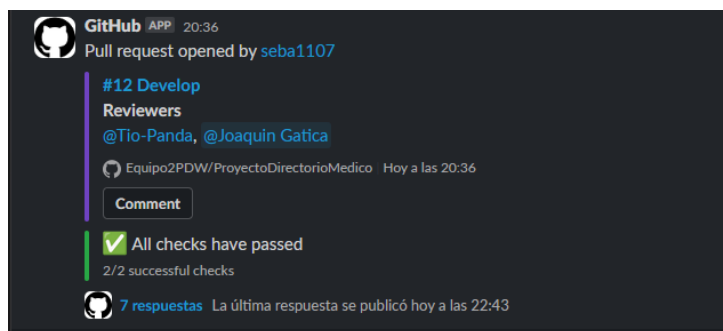


Gráfico 8: CI al momento de realizar un push a develop.

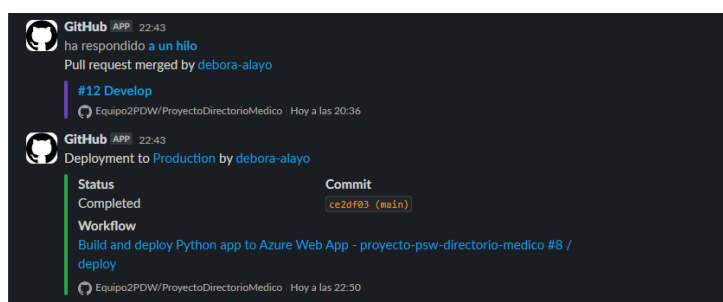


Gráfico 9: CD al momento de realizar un PR a main.

6. Problemas encontrados y soluciones

Para esta entrega, la dificultad principal fue aprender la documentación de Github Actions, ya que se ingresan los conceptos de "job", "steps", etc. Más que esto, la implementación de las nuevas funcionalidades de la aplicación tuvieron mucha menos dificultad con respecto a la entrega anterior, ya que adelantamos un poco de trabajo.